

What was quietly broken, and what we did about it

A full review of the NERVIS ecosystem — the system that routes your AI requests, measures which models are any good, and shows you what happened. Written to be readable without knowing the code.

THE SHORT VERSION

The project had a habit of **telling you things that were not true** — a screen saying "no data" while data existed, a button that looked ready and did nothing, a bill that left calls out. Not one of these was caught by its own tests, because the tests were passing the entire time.

We audited all four parts of the system against what they *actually did* when running, found **63 confirmed problems**, and fixed every one — including a security hole that would have let a stranger change your provider keys.

1 Why we did this

You said it plainly: "*I keep seeing cards that should work, but don't — not exactly confidence inspiring about the whole project, no?*"

That was the right question, and the honest answer turned out to be worse than the symptom. Over one day, the same defect had been found **eight separate times** — always by you or me looking at a screen, never by the project's own checks. Each time it was fixed as a one-off. Eight one-offs is not bad luck; it is a signal that something underneath is wrong.

So instead of fixing a ninth, we audited the whole system. Not by reading the code and trusting it, but by **running it and comparing what it claimed with what it did**. Eight independent reviews ran in parallel, each covering a different area. Every problem one of them raised was then handed to a second reviewer whose job was to *disprove* it — findings that could not be verified were thrown out.

Why the tests did not catch any of this

This is the uncomfortable part, and it is worth understanding because it explains the whole report. The project has around 1,400 automated tests and they were **all passing** throughout — before the audit and after it.

Tests only check what somebody thought to check. Several here were written at the same moment as the code they test, so they recorded what the code *did* rather than what it was *supposed to do*. When the code was wrong, the test locked the mistake in place and went green forever. A few examples we found:

- A test literally named "*ingestion answers 202*" that checked for the number **200** — the wrong answer, asserted as correct.
- A test proving a part of the system "accepts no changes" that only tried one kind of change. The kind it did not try had been shipping for weeks.
- A stored "expected result" file containing a **completely invented** measurement — for the case that means "nothing was measured".

Passing tests were never evidence for this class of problem. Running the real thing was.

2 What the audit found

63

problems confirmed
after double-checking

22

serious enough to
mislead or cost money

1

security hole
(now closed)

63

fixed
— all of them

The problems fell into a few recognisable shapes. Almost none were "the code crashes"; nearly all were **the system saying something untrue about itself**.

SHAPE OF PROBLEM	WHAT IT LOOKED LIKE TO YOU	COUNT
Mislabelled data	Real, live numbers shown greyed-out as "placeholder" — or invented placeholder numbers shown as if they were real.	14
Descriptions that expired	Text explaining that a feature "isn't built yet" — for features that had been working for weeks.	13
Tests that could not fail	Checks that appeared to guard something but tested the wrong thing, or nothing at all.	10
"Empty" read as "broken"	A service answering "I have nothing yet" treated as "the service is down" — and replaced with made-up data.	5
Wrong answers	Money miscounted, models sorted into the wrong groups, records filed under the wrong labels.	11
Things that did nothing	Buttons that looked ready and threw an error on click; a guard that never guarded.	10

THE SINGLE MOST IMPORTANT FINDING

Those eight repeated "cards that should work" were **not eight mistakes**. They were one.

Every screen fetches data through one shared piece of code. That code *knew* whether the service had answered — and then threw that knowledge away before handing the data on. So every screen had to guess, and each one guessed differently. Ten different guessing methods had grown up around the gap, and **68 screens gave up guessing entirely** and hardcoded an answer.

Making that one function pass along the answer it already had fixed the whole class.

3 The problems that mattered most

SECURITY · FIXED

A stranger could have changed your provider keys

The system has a rule: if it is reachable from outside this machine, an unidentified caller must not be allowed to change stored API keys. The code for that rule existed and read well.

It asked the caller a question the caller had no way of answering — the equivalent of checking for a field on a form that the form does not have. A missing answer was treated as "*fine, go ahead*". The guard never once refused anybody.

In practice: on a machine exposed to a network, anyone who could reach it could add, delete or redirect the keys that pay for your AI usage.

Fixed. The check now asks a question that exists, and asks it in a way that raises an error if anyone renames it. We also swept the whole codebase for the same pattern — no others.

WRONG ANSWERS · FIXED

Cloud models were being labelled as running on your own machine

Two of the routing groups exist entirely to answer "where does this request go?" — one for models on your machine, one for models in the cloud. The screen had them **exactly backwards**: it listed 49 cloud models (Anthropic's and Google's) as local, and showed the cloud group as empty.

Worse, the group named **private** — the one whose entire promise is that nothing leaves your computer — was listing Anthropic and Google as its members.

The same system already answered this correctly elsewhere. One service was giving two opposite answers about the same list, depending on which part you asked.

Fixed, and confirmed against the running system: the cloud group now holds the cloud models, and the local and private groups correctly show empty when no local model is running.

MONEY · FIXED

Ordinary requests were never counted toward what you spend

There are four ways a request can complete. Three of them recorded what the call cost. The fourth — the plainest, most common kind — received the provider's own token counts, forwarded them to you, and **threw the copy away** without recording anything.

Those calls were invisible: absent from your usage totals, absent from the per-call table, and — most importantly — absent from the budget limits that are supposed to stop spending.

No test caught it because the fake provider used in testing returned a response with the cost information **missing entirely** — a shape no real provider produces. A test fixture that cannot carry the field cannot notice when the field is dropped.

Fixed, and the fake now returns what a real provider returns.

MONEY · FIXED

Anthropic calls using cached text were billed as if the new text were free

Providers report token usage in different formats. Most give a total with the cached portion listed inside it. Anthropic reports the two *separately*. Our code assumed everyone used the first format and subtracted the cached amount from the total.

For Anthropic that subtracted something that was never added. A request with 20 new words against 5,000 cached ones came out as **zero new words** — priced as free.

Always in the same direction: under-counting. A budget reads an undercount as money still available.

Fixed, and the assumption that was never written down anywhere is now written down.

MISLEADING DISPLAY · FIXED

"No trace has been recorded yet" — on a system with recorded traces

A trace is the record of one request's journey. The screen that lists them asked for "the most recent" — and the code underneath handed back **the oldest** ones it had ever stored. Past a certain point, everything recent became invisible.

The same fault hit the event feed: the "Recent events" panel showed the four *oldest* events in the system, which never change. Nothing from that day was reachable.

WORTH ADMITTING

My first fix for this was wrong. I made it read the newest *events* instead of the newest *traces* — so after a restart, when recent activity was all routine background chatter, it reported **zero traces** while three existed. I had moved the problem from one end to the other and nearly called it done. The live system caught me within minutes of restarting it.

Fixed properly: it now asks for the most recent traces, not the most recent events. Verified live — three traces, newest first.

4 Things that looked like they worked

DEAD CONTROLS · FIXED

Two buttons that did nothing at all

The benchmark screen's main button — the big one, the primary action — called a function that **does not exist**. Clicking produced an error invisible to you: no message, no reaction, nothing. "Clear filters" had the same fault for a different reason.

There is a helper here whose entire purpose is to prevent this: it makes either a working button or a visibly greyed-out one explaining what it waits for. Both had been written *around* it.

Fixed, and there is now an automatic check that refuses any button pointing at a function that does not exist. It found the second button — I had only been looking for the first.

DEAD CONTROLS · FIXED

A "Release" button that could never release

When you release a model holding memory, and that slot holds more than one, the system asks you to confirm. The confirmation referred to a value that does not exist, so it failed **before the question could appear**. No dialogue, no action, no error.

It only failed in the multi-model case — which is the exact case the confirmation was written for. The single-model path worked, so the button looked fine.

FRESH INSTALL · FIXED

Six screens went completely blank for a brand-new user

Three states have to work: nothing running, everything running, and — the one nobody had tested — **everything running but empty**, which is exactly what a new user sees on day one. In that state six of the 35 screens crashed and rendered nothing at all. Not an empty table — a blank page.

Fixed, and this is now an automatic check. It immediately caught a mistake *I* had made an hour earlier in the same session.

HONESTY OF MEASUREMENTS · FIXED

Benchmarks that blamed the model for the computer's problems

When testing whether a model can use tools correctly, any crash of the underlying software — a disconnect, a timeout, the model host falling over — was recorded as **"the model refused to use the tool"**. That verdict is stored as a *measurement*, and the router reads it to decide whether a model is good enough for coding work.

A model could be disqualified because your computer hiccuped.

Fixed. A crashed run is now recorded as a crashed run, and produces no score at all rather than a bad one.

Benchmarks taken on a struggling machine were marked "valid"

This laptop loses roughly **48% of its speed** when hot, so the system flags measurements taken under heat. It only noticed heat that *changed* during a run — so a test that was hot from beginning to end had one consistent reading, nothing was flagged, and the results were published as clean.

Same for memory: the system recorded whether it ran out of memory at four points during every benchmark and never once looked at those readings. A benchmark that spent its time waiting on the hard disk was published as a clean speed measurement. **Both are now flagged.**

5 The labels on your screens

This is where the audit started, and the numbers moved furthest.

Every panel on the dashboard is meant to show whether its numbers are **real** (read live from a running service) or a **placeholder** (an example, shown greyed out and marked). Getting that label wrong is worse than showing nothing: you either dismiss real information as fake, or trust invented information.

MEASURE	BEFORE	AFTER
Panels physically unable to report whether their data is real	88	55
Panels showing real data labelled as placeholder	17	1 (<i>explained below</i>)
Panels showing made-up data labelled as real	14	0

The one remaining is honest and understood: a screen listing downloadable models mixes a stored catalogue (a fixed list) with one live figure (your free disk space). Labelling the whole panel either way would be wrong, so it stays as it is, documented.

The 55 panels that still cannot report are not bugs — they are things like empty states, reference tables and explanations, where there is no data to be real or fake. Forcing them to claim otherwise would be the same dishonesty pointed the other way.

Two examples of what this looked like

- "Evidence in the ranking" displayed 32 rows of genuine benchmark results from your own machine — greyed out, marked as an example. The information needed to label it correctly had been sitting unused in the data since the day it was written.
- The **system information screen** showed a made-up computer — a fictional machine ID, invented memory figures — labelled as "*sampled now*".

THE CHECKER WAS ITSELF HALF-BLIND

I built a tool to catch these: it draws every screen twice, once with the services off and once with them on, and compares. It found a lot — but it silently skipped any panel whose title contains live data (like "Machine mc_7a1f"), because the title differed between the two runs and it could not match them up.

Those were exactly the panels most worth checking. Once fixed, **six more** mislabelled panels appeared immediately.

6 What now stops this coming back

Fixing 63 problems is worth less than making the 64th impossible to ship. Most of the work went here.

NEW AUTOMATIC CHECK	WHAT IT REFUSES TO LET THROUGH
Honesty check	Draws all 35 screens twice — services off, then on — and compares every label against where the data actually came from. Findings must survive a second run to be reported, so it does not cry wolf.
Fresh-install check	Draws all 35 screens against services that are running but hold nothing — the day-one experience. Six screens used to crash here.
Label ratchet	Counts panels that cannot report where their data came from. The number may go down and may never go up.
Dead-control check	Refuses any button pointing at a function that does not exist. Found one I had missed.
Syntax trap check	Catches a specific mistake that broke the dashboard four times in one day while I was writing these fixes.

Beyond the checks, three fixes were made at the **source** rather than the symptom, which is what stops a family of bugs rather than one bug:

- The shared data-fetching function now passes along whether the service answered — closing the gap that produced eight identical bugs.
- A panel builder that **refuses to build** a panel that has not stated whether its data is real. Forgetting is now a build failure, not a wrong label.
- Names for things that were previously typed out by hand in several places, where a typo silently disabled a safety check.

And the tests

We added tests for the code the existing suite was walking past — including the function that every streamed price depends on, which had been running **three of its twenty-six lines** and never once reached the part that reads the numbers. Every fix in this report was verified the same way: **break it again on purpose**, confirm the new test fails, then restore the fix and confirm it passes. A test that has never failed has not been shown to work.

7 Honest notes

Things that went wrong during the work itself. They belong in the report because they are the same kinds of mistake it is about.

I broke your provider list

Partway through, OpenRouter disappeared from your providers. That was me: restarting the services from a terminal that did not have your provider list configured. Your key was never lost — but the restart brought the system up **healthy, authenticated, and quietly smaller**, and nothing said a word about it.

That is a genuine design gap, not just my mistake: the system remembers your *key* and whether a provider is switched on, but not *which providers exist*. The launcher now warns you when it finds a stored key for a provider it is not starting, and prints the exact line to fix it.

My first fix for the traces screen was wrong

Described in section 3. I moved a problem from one end to the other and would have reported it fixed. Restarting the real system caught it in under a minute — which is the whole argument of this report in miniature.

One of my own tests could not fail

I wrote a test to prove a search bug was fixed. It picked which record to look for in advance, that record happened to sort first, and so it **passed against the very bug it was written for**. Exactly the fault the audit had just found in three other tests. It now chooses its target after the data exists, so it cannot be satisfied by luck.

Twenty minutes lost to a stale cache

After deliberately re-breaking something to prove a test caught it, Python's cached copy of the old code outlived the restore. I spent twenty minutes proving a file was correct while the computer was running the previous version of it.

WHERE THIS LEAVES THINGS

All 63 confirmed findings are fixed. 1,477 automated tests pass, every quality check is green, and the fixes that could be confirmed against the running system were.

The thing I would not claim is that the system is now correct. What changed is the *basis* for saying so: before this, "the tests pass" was the evidence, and that evidence had been wrong for a long time without anybody noticing. Now several classes of untruth are caught mechanically, and the rest were checked against a system that was actually running.